GPT-6 Astra API - Try OpenAI GPT-6 on Kie AI

Production-Ready Implementation Plan

1. Product Target

Build a multi-tenant commerce operating system with:

- Public tenant storefronts
- Private tenant dashboards
- Platform administration
- Configurable business capabilities
- Retail, food ordering, services, bookings, quotes, and inquiries
- Product, service, customer, order, inventory, payment, marketing, and analytics workflows
- AI-assisted operations using approved tools and structured actions
- Branded subdomains and verified custom domains
- 30-day trials with recurring billing
- Real-time notifications and operational updates
- Strong tenant isolation
- Auditable, reversible, and observable business operations
- Production-grade deployment, monitoring, backups, disaster recovery, and security controls

The system must be implemented as a stable shared runtime with versioned configuration and modular capabilities. Tenants must not receive generated production code. The platform should generate and manage validated business configurations, not arbitrary application code.

---

2. Non-Negotiable Production Standards

The platform is not production-ready unless all of the following are true:

- Tenant isolation is enforced in application logic, database policies, storage, events, jobs, exports, and AI tools.
- All financial operations are idempotent and auditable.

- Payment status is derived from verified provider webhooks, not browser redirects.
- Every important state transition is explicit, validated, logged, and recoverable.
- Configuration changes support drafts, previews, publishing, version history, and rollback.
- AI actions are constrained to approved tools and validated schemas.
- Background jobs are retry-safe and tenant-aware.
- Customer data can be exported and deleted according to policy.
- Backups are encrypted and restore-tested.
- Deployment is automated with approval gates for production.
- Security testing includes automated and human review.
- Operational incidents have runbooks and assigned ownership.
- The system supports graceful degradation when third-party services fail.
- All APIs return consistent errors, request IDs, and traceable audit information.
- No critical or high-risk security findings remain unresolved before public launch.

---

3. Recommended Final Architecture

3.1 Repository Structure

Use a monorepo managed with Turborepo or Nx.

```
apps/
  marketing/
  storefront/
  dashboard/
  admin/
  api/
  worker/
  realtime/

packages/
  ui/
  design-tokens/
  database/
  db-client/
```

```
  auth/
  authorization/
  tenant-context/
  tenant-runtime/
  configuration/
  validation/
  billing/
  payments/
  orders/
  catalog/
  inventory/
  bookings/
  customers/
  promotions/
  quotes/
  messaging/
  analytics/
  notifications/
  storage/
  search/
  ai/
  events/
  jobs/
  email/
  observability/
  security/
  exports/
  testing/
  feature-flags/
  logger/

infra/
  terraform/
  kubernetes/
  docker/
  migrations/
  monitoring/
  runbooks/
  policies/
```

```
docs/
  architecture/
  adr/
  api/
  security/
  compliance/
  operations/
```

3.2 Applications

Marketing Application

Responsibilities:

- Product marketing
- Pricing
- Industry pages
- Template previews
- Documentation
- Help center
- Legal documents
- Signup and login entry points
- Public status information
- Contact and support forms

Routes:

```
/
/solutions/retail
/solutions/restaurants
/solutions/services
/solutions/bookings
/solutions/quotes
/templates
/pricing
/security
```

```
/help
/status
/legal/privacy
/legal/terms
/login
/start
```

Storefront Application

Responsibilities:

- Hostname resolution
- Tenant configuration loading
- Public page rendering
- Catalog browsing
- Search
- Cart
- Checkout
- Customer authentication
- Order confirmation
- Booking flows
- Quote requests
- Customer messaging
- SEO metadata
- Structured data
- Promotions
- Abandoned-cart handling

Dashboard Application

Responsibilities:

- Tenant onboarding

- Operational dashboard
- Catalog
- Orders
- Customers
- Inventory
- Bookings
- Quotes
- Promotions
- Analytics
- Appearance
- Staff permissions
- Billing
- Exports
- AI assistant
- Audit history

Platform Admin Application

Responsibilities:

- Tenant administration
- Subscription monitoring
- Domain and SSL status
- Usage and quotas
- Support tools
- Audited impersonation
- Failed jobs
- Webhook reconciliation
- Security alerts

- AI action review
- Account suspension
- Data retention operations
- Platform analytics

API Application

Responsibilities:

- Versioned REST or RPC APIs
- Authentication validation
- Authorization
- Request validation
- Idempotency
- Rate limiting
- API error envelopes
- Request tracing
- Webhook endpoints
- Public checkout endpoints
- Internal service APIs

Worker Application

Responsibilities:

- Background jobs
- Webhook processing
- Email and SMS delivery
- Catalog imports
- AI parsing
- Image processing
- Search indexing

- Analytics aggregation
- Billing reminders
- Domain verification
- Data exports
- Cleanup and retention jobs

Realtime Application

Responsibilities:

- Tenant-scoped subscriptions
- Dashboard order updates
- Inventory changes
- Booking changes
- Notification delivery
- Job progress updates
- AI action status
- Presence where necessary

---

4. Technology Baseline

4.1 Frontend

- Next.js with TypeScript
- React Server Components where appropriate
- Tailwind CSS
- Custom design system
- React Hook Form
- Zod validation
- TanStack Query for client-side server state
- Playwright for end-to-end testing

- Storybook for shared UI components

Avoid coupling core workflows entirely to client-side state. Server-side authorization and validation must remain authoritative.

4.2 Backend

Use a modular service-oriented backend within a shared repository.

Recommended approach:

- Next.js for marketing and storefront rendering
- Dedicated API service for business APIs
- Dedicated worker service for asynchronous jobs
- Dedicated realtime gateway
- Domain modules with explicit service boundaries
- PostgreSQL as the system of record

Do not split into many independently deployed microservices too early. Use modular services first, with independently deployable workers and realtime infrastructure. Extract services only when operational or scaling requirements justify it.

4.3 Infrastructure

Recommended production baseline:

- Managed PostgreSQL with high availability
- Read replica for reporting workloads
- Redis-compatible cache and queue backend
- S3-compatible object storage
- CDN and edge caching
- Managed container platform or Kubernetes
- Secret manager
- Managed DNS and certificate automation
- Stripe for billing and payments

- Transactional email provider
- SMS provider
- Error tracking
- Centralized logs
- Metrics and tracing
- Managed WAF and DDoS protection

All infrastructure must be reproducible through Terraform or an equivalent infrastructure-as-code system.

---

5. Environment Strategy

Create separate environments:

```
local
development
test
preview
staging
production
disaster-recovery
```

5.1 Local

- Docker Compose
- Local PostgreSQL
- Local Redis
- Local object storage emulator
- Mock payment provider
- Seed tenants and test users
- Local mail capture
- Local tracing and logs

5.2 Development

- Shared developer environment
- Non-production integrations
- Safe test payment credentials
- Disposable tenant data
- Automatic cleanup for temporary resources

5.3 Preview

Every pull request receives:

- Isolated URL
- Separate database schema or database
- Seeded Tenant A and Tenant B
- Owner, manager, read-only, and fulfillment users
- Mock payment data
- Storage fixtures
- Realtime fixtures
- Test custom-domain simulation

5.4 Staging

Staging must closely match production:

- Same deployment artifacts
- Same database engine version
- Same queue behavior
- Same authorization configuration
- Same observability
- Provider test-mode integrations
- Synthetic monitoring
- Restore testing

5.5 Production

Production must use:

- Separate credentials
- Private network access for databases
- Managed secrets
- Database backups
- Point-in-time recovery
- Multi-zone deployment
- WAF
- Rate limiting
- Audit logging
- Alerting
- Deployment approval gates

---

6. Domain Model

6.1 Platform Identity

```
users
user_sessions
user_devices
user_mfa_methods
user_recovery_codes
user_consents
```

6.2 Tenancy

```
organizations
organization_members
organization_roles
organization_permissions
organization_settings
organization_features
```

```
organization_domains
organization_usage
organization_limits
organization_audit_events
organization_configurations
organization_configuration_versions
organization_releases
```

Every tenant-owned record must contain:

```
organization_id uuid not null
```

Tenant ownership must be immutable after creation.

6.3 Catalog

```
catalogs
categories
products
product_variants
product_options
product_option_values
product_images
product_documents
product_prices
product_tax_categories
product_status_history
```

Products should support:

- Physical products
- Digital products
- Services
- Bookable services
- Menu items
- Quote-only offerings

Do not create entirely separate product systems for each business mode. Use a shared catalog model with capability-specific extensions.

6.4 Inventory

```
inventory_locations
inventory_items
inventory_levels
inventory_reservations
inventory_movements
inventory_adjustments
inventory_transfers
inventory_alerts
```

Inventory quantities must use transactional updates and database constraints. Prevent negative inventory unless explicitly configured for a tenant.

6.5 Commerce

```
carts
cart_items
checkout_sessions
orders
order_items
order_addresses
order_events
order_notes
fulfillments
fulfillment_items
shipments
returns
refunds
```

6.6 Payments

```
payment_customers
payment_methods
payment_intents
payments
payment_events
```

```
refunds
payout_records
payment_reconciliation_items
```

Never store raw card data.

6.7 Customers

```
customers
customer_addresses
customer_consents
customer_notes
customer_segments
customer_events
customer_sessions
customer_preferences
```

6.8 Bookings

```
services
staff_profiles
resources
availability_rules
availability_exceptions
appointment_slots
appointments
appointment_events
booking_policies
booking_waitlists
```

6.9 Promotions

```
discounts
discount_codes
discount_rules
discount_redemptions
campaigns
campaign_events
```

6.10 Quotes and Inquiries

```
quotes
quote_items
quote_events
inquiries
inquiry_messages
inquiry_assignments
```

6.11 AI

```
ai_conversations
ai_messages
ai_requests
ai_tool_calls
ai_actions
ai_action_previews
ai_action_approvals
ai_usage
ai_failure_events
ai_policy_decisions
```

6.12 Operations

```
jobs
job_attempts
outbox_events
webhook_events
notification_deliveries
email_deliveries
sms_deliveries
exports
export_files
imports
import_rows
security_events
```

---

7. Database and Tenant Isolation

7.1 PostgreSQL RLS

Enable RLS on every tenant-exposed table.

Use a database session variable set by trusted server code:

```
select set_config(
  'app.current_organization_id',
  'organization-uuid',
  true
);
```

Policies must validate the session organization and active membership.

```
create policy organization_orders_select
on orders
for select
to authenticated
using (
  organization_id = current_setting(
    'app.current_organization_id',
    true
  )::uuid
);
```

For writes:

```
create policy organization_orders_insert
on orders
for insert
to authenticated
with check (
  organization_id = current_setting(
    'app.current_organization_id',
    true
  )::uuid
);
```

Do not rely exclusively on client-supplied tenant IDs.

7.2 Authorization Layers

Every protected operation must pass:

1. Session authentication

2. Organization resolution
3. Active membership validation
4. Role and permission validation
5. Resource ownership validation
6. Business-rule validation
7. Database RLS enforcement
8. Audit logging where applicable

7.3 Database Rules

- Tenant IDs cannot be updated.
- Foreign keys must include tenant ownership where practical.
- Use composite unique indexes such as `(organization_id, slug)`.
- Use database constraints for valid state values.
- Use transactions for financial and inventory operations.
- Avoid soft deletion without clear retention behavior.
- Use immutable event tables for financial and audit history.
- Add indexes for every high-volume tenant query.
- Partition high-volume global event tables if required.

---

8. Tenant Lifecycle

Implement an explicit tenant state machine:

```
onboarding
setup_incomplete
ready_to_publish
active
trialing
trial_expiring
payment_required
past_due
grace_period
```

```
restricted
canceled
suspended
closed
deletion_pending
deleted
```

Transitions must be:

- Explicit
- Validated
- Audited
- Idempotent
- Triggered by commands or verified provider events
- Recoverable

Each transition should create:

```
organization_status_events
```

Include:

- Previous state
- New state
- Actor
- Reason
- Source
- Request ID
- Timestamp
- Metadata

---

9. Authentication and Identity

Implement:

- Passwordless email login
- Optional password login
- Passkeys
- OAuth providers
- MFA for owners and administrators
- Recovery codes
- Session revocation
- Device management
- Suspicious login detection
- Email verification
- Account recovery
- Reauthentication for sensitive actions

Sensitive operations require recent authentication:

- Changing payment settings
- Issuing refunds
- Changing staff permissions
- Exporting customer data
- Changing domains
- Deleting the organization
- Creating API keys
- Modifying webhooks
- Changing billing plans

Use secure, HTTP-only cookies for dashboard sessions. Avoid storing long-lived authentication tokens in local storage.

---

10. Roles and Permissions

Define permissions independently from UI navigation.

10.1 Default Roles

```
owner
administrator
manager
sales_staff
fulfillment_staff
inventory_staff
finance_staff
marketing_staff
read_only_analyst
platform_support
platform_administrator
```

10.2 Permission Examples

```
catalog.product.read
catalog.product.create
catalog.product.update
catalog.product.publish
orders.read
orders.update
orders.refund
customers.read
customers.export
inventory.adjust
bookings.manage
billing.read
billing.manage
staff.invite
staff.permission_manage
settings.manage
domain.manage
ai.execute_low_risk
ai.approve_customer_facing
ai.approve_financial
```

Authorization should be tested independently of route visibility.

11. Configuration and Publishing System

Configuration categories:

```
brand_config
layout_config
navigation_config
catalog_config
checkout_config
order_config
booking_config
notification_config
seo_config
feature_flags
custom_css_tokens
```

Every configuration has:

```
draft
preview
published
archived
```

Publishing flow:

```
Edit draft
  -> Validate schema
  -> Run business rules
  -> Generate preview release
  -> Run automated checks
  -> Show preview
  -> Approve publish
  -> Create immutable release
  -> Update active release pointer
  -> Purge relevant caches
  -> Emit organization.published
```

Required capabilities:

- Version history

- Rollback
- Change author
- Manual versus AI-generated source
- Validation errors
- Preview URLs
- Release notes
- Audit events
- Safe cache invalidation

Never render arbitrary unvalidated JSON.

Use versioned schemas:

```
theme_config_v1
theme_config_v2
checkout_config_v1
booking_config_v1
```

---

12. Storefront Runtime

12.1 Request Resolution

```
Incoming hostname
  -> Edge middleware
  -> Normalize hostname
  -> Resolve domain record
  -> Resolve organization
  -> Verify domain status
  -> Load published release
  -> Load enabled capabilities
  -> Load public content
  -> Render template
  -> Apply theme tokens
  -> Set cache policy
```

12.2 Domain Types

Support:

- Platform subdomains
- Custom subdomains
- Apex custom domains
- Multiple domains per tenant
- One primary domain
- Redirect domains
- Domain verification
- SSL provisioning
- Domain transfer warnings
- Deleted-slug quarantine

Never activate a custom domain before ownership verification.

12.3 Templates

Launch the final platform with four distinct template families:

- Minimal
- Boutique
- Modern Grid
- Service Studio

All templates consume the same content and commerce contracts. Template changes must not migrate or mutate business data.

12.4 Storefront Requirements

- Responsive layouts
- Accessibility conformance
- SEO metadata
- Open Graph metadata

- Schema.org structured data
- Sitemap generation
- Robots configuration
- Product canonical URLs
- Fast image delivery
- CDN caching
- Cart persistence
- Customer account support
- Error pages
- Maintenance and restricted-state pages
- Localization readiness
- Currency formatting
- Time-zone-aware booking display

---

13. Catalog and Import System

Supported import sources:

- Manual entry
- CSV
- Spreadsheet
- PDF
- Image
- Public website
- API import

Pipeline:

```
Upload
  -> Malware scan
  -> MIME and extension validation
```

```
-> File size and page limits
-> Text/image extraction
-> AI parsing
-> Schema validation
-> Duplicate detection
-> Human review
-> Draft records
-> Manual approval
-> Publish
```

Requirements:

- Never automatically publish AI-imported catalog records.
- Store original uploaded files.
- Preserve extraction confidence.
- Show row-level validation errors.
- Allow corrections before import.
- Provide rollback for completed imports.
- Make imports resumable.
- Prevent duplicate processing with idempotency keys.
- Enforce tenant-scoped storage paths.
- Scan files for malware.
- Reject unsupported content safely.

---

14. Commerce and Order Processing

14.1 Money

Store all money as integer minor units:

```
amount_minor_units bigint
tax_minor_units bigint
discount_minor_units bigint
shipping_minor_units bigint
currency char(3)
```

Never use floating-point arithmetic for financial values.

14.2 Order State Machine

```
draft
pending_payment
paid
confirmed
processing
ready
fulfilled
completed
canceled
refunded
partially_refunded
payment_failed
```

Each transition must:

- Verify the current state
- Validate actor permissions
- Validate business rules
- Run inside a transaction
- Create an order event
- Emit an outbox event
- Be idempotent

14.3 Checkout

Checkout must:

- Recalculate prices server-side
- Revalidate product availability
- Recalculate tax
- Revalidate discounts
- Validate shipping or pickup method

- Create an idempotency record
- Create a payment intent
- Prevent client-side price tampering
- Handle retries safely
- Avoid duplicate orders

The browser success page is not payment proof.

---

15. Payments and Billing

15.1 Payment Webhooks

Webhook handling:

```
Receive request
  -> Verify signature
  -> Store raw payload
  -> Check event ID
  -> Mark received
  -> Enqueue processing job
  -> Process transactionally
  -> Record outcome
  -> Retry if safe
  -> Alert after permanent failure
```

Use an inbox table for webhook deduplication and an outbox table for emitted events.

15.2 Subscription States

Maintain an internal state machine synchronized with the provider:

```
trialing
trial_expiring
payment_required
active
past_due
grace_period
restricted
```

```
canceled
suspended
closed
```

Implement:

- 30-calendar-day trial
- Payment method verification
- Trial reminders at 7 days, 3 days, and 24 hours
- Clear renewal price
- Clear renewal date
- One-click cancellation
- Grace period
- Read-only dashboard after suspension
- Storefront disablement rules
- Export period after cancellation
- Billing reconciliation
- Provider event replay
- Manual support correction tools

15.3 Financial Controls

- No direct database edits to financial records
- Refunds require explicit authorization
- Refund idempotency
- Payment reconciliation jobs
- Immutable payment events
- Daily reconciliation reports
- Alerts for mismatched states
- Provider dashboard access restricted

- Full audit trail for billing changes

---

16. Bookings

Booking functionality must support:

- Services
- Staff
- Resources
- Time zones
- Business hours
- Availability rules
- Exceptions
- Buffer times
- Capacity
- Deposits
- Cancellations
- Rescheduling
- No-shows
- Confirmation messages
- Reminder messages
- Waitlists
- Conflict detection

Booking creation must use a transaction or locking strategy that prevents double booking.

---

17. Real-Time Architecture

Use tenant-scoped realtime channels.

Example channels:

```
organization:{organization_id}:orders
organization:{organization_id}:inventory
organization:{organization_id}:bookings
organization:{organization_id}:notifications
organization:{organization_id}:jobs
organization:{organization_id}:ai-actions
```

Realtime authorization must validate:

- Authenticated user
- Active membership
- Organization ownership
- Permission for the event stream
- Suspended or closed tenant state

Realtime events must not expose records from other tenants. Test realtime authorization separately from ordinary database queries.

Use database changes or domain events as the source for realtime updates. Do not broadcast uncommitted state.

---

18. AI Platform

18.1 AI Design

The AI system should operate through approved tools only:

```
list_products
create_product_draft
update_product_draft
create_category_draft
update_store_theme
list_orders
summarize_sales
create_discount_draft
draft_customer_reply
generate_policy_draft
```

```
suggest_missing_images
detect_duplicate_products
```

The model must never:

- Execute raw SQL
- Modify arbitrary database tables
- Bypass permission checks
- Publish financial changes without approval
- Change staff permissions without owner approval
- Export customer data without reauthentication
- Modify payment destinations autonomously
- Deploy application code

18.2 Action Classification

```
read_only
low_risk_write
customer_facing_write
financial_action
security_sensitive_action
```

Approval requirements:

- Read-only: immediate
- Draft creation: usually immediate
- Customer-facing changes: user approval
- Refunds and subscription changes: explicit confirmation
- Staff permissions: owner approval
- Customer export: reauthentication
- Payment destination changes: owner approval plus verification

18.3 Action Contract

```
{
  "action_type": "update_product",
  "organization_id": "uuid",
  "actor_user_id": "uuid",
  "before": {},
  "proposed_after": {},
  "validation_errors": [],
  "requires_approval": true,
  "status": "pending"
}
```

Every action must store:

- Prompt or request
- Model and version
- Tool name
- Input
- Output
- Validation result
- Before state
- Proposed after state
- Approval actor
- Execution result
- Rollback reference
- Token and cost usage
- Failure information

---

19. Background Jobs

Every job must include:

```
job_id
organization_id
job_type
```

```
status
attempt_count
started_at
completed_at
last_error
next_retry_at
idempotency_key
```

Required job categories:

- Catalog parsing
- File scanning
- Image optimization
- Search indexing
- Email delivery
- SMS delivery
- Subscription reminders
- Trial expiration
- Payment reconciliation
- Webhook retries
- Analytics aggregation
- Backup verification
- Domain verification
- Security scans
- AI processing
- Data exports
- Data deletion
- Retention cleanup

Implement:

- Exponential backoff

- Dead-letter queues
- Maximum attempts
- Tenant-aware authorization
- Job cancellation
- Job progress
- Duplicate prevention
- Operator retry tools
- Failure alerts
- Correlation IDs

---

20. Notifications

Channels:

- In-app
- Email
- SMS
- Push
- WhatsApp where legally and operationally appropriate

Notification requirements:

- User preferences
- Tenant-level defaults
- Frequency controls
- Unsubscribe handling
- Delivery status
- Retry behavior
- Template versioning
- Localization support

- Consent records
- Suppression lists
- Provider webhook processing

Notifications must be event-driven rather than embedded directly inside request handlers.

---

21. Analytics

21.1 Tenant Analytics

Measure:

- Revenue
- Orders
- Average order value
- Conversion rate
- Returning customers
- Top products
- Inventory turnover
- Booking utilization
- Abandoned carts
- Refund rate
- Customer growth

21.2 Platform Analytics

Measure:

- Onboarding completion
- Time to first publish
- Time to first product
- Time to first order

- Trial activation
- Trial-to-paid conversion
- Churn
- Feature activation
- AI usage
- AI acceptance rate
- Import correction rate
- Support burden
- Failed payments
- Checkout success
- Job failure rate
- Webhook latency

Keep tenant analytics isolated from platform analytics. Minimize personal data collection.

---

22. API Standards

All APIs must provide:

- Versioned routes
- Zod or equivalent validation
- Authentication
- Authorization
- Rate limiting
- Request IDs
- Idempotency keys where applicable
- Cursor pagination
- Consistent error envelopes
- Audit events

- Structured logs
- OpenAPI documentation

Response format:

```
{
  "data": {},
  "error": null,
  "request_id": "req_123"
}
```

Error format:

```
{
  "data": null,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "The order could not be found.",
    "field_errors": []
  },
  "request_id": "req_123"
}
```

Do not leak internal database errors, stack traces, secrets, or tenant existence through error messages.

---

23. Event-Driven Architecture

Use domain events:

```
organization.created
organization.published
organization.suspended
product.created
product.updated
product.published
order.created
order.paid
order.fulfilled
```

```
customer.created
booking.created
payment.failed
subscription.trial_ending
subscription.renewed
subscription.past_due
domain.verified
ai.action.approved
export.completed
```

Use the transactional outbox pattern:

```
Business transaction
  -> Write domain record
  -> Write outbox event
  -> Commit transaction
  -> Worker publishes event
  -> Mark event published
```

Events must include:

```
event_id
event_type
organization_id
aggregate_type
aggregate_id
occurred_at
request_id
actor_id
payload_version
payload
```

Consumers must be idempotent.

---

24. Security Program

24.1 Security Baseline

Use OWASP ASVS as the application security baseline.

Security controls:

- TLS everywhere
- Encryption at rest
- Managed secret storage
- No secrets in client bundles
- Secure cookies
- CSRF protection
- Content Security Policy
- Strict transport security
- Input validation
- Output encoding
- Rate limiting
- WAF
- Malware scanning
- Signed private file URLs
- Audit logs
- Session revocation
- Dependency scanning
- Secret scanning
- Container scanning

24.2 Required Tests

- Broken access control
- IDOR
- SQL injection
- XSS
- CSRF

- SSRF
- File-upload abuse
- Rate-limit bypass
- Webhook replay
- Session fixation
- Privilege escalation
- Cross-tenant storage access
- Cross-tenant realtime access
- Payment tampering
- Export authorization
- AI tool authorization
- Background-job isolation
- Suspended tenant restrictions
- Deleted-user session invalidation

Do not rely exclusively on automated pentesting. Perform threat modeling and external penetration testing.

---

25. Storage Security

Storage paths must be tenant-scoped:

```
organizations/{organization_id}/assets/{asset_id}/original
organizations/{organization_id}/assets/{asset_id}/optimized
organizations/{organization_id}/exports/{export_id}
```

Requirements:

- Private by default
- Signed URLs
- Object ownership validation
- MIME validation

- File-size limits
- Malware scanning
- Image re-encoding
- Metadata stripping
- Retention policies
- Access logging
- Tenant isolation tests
- No user-controlled unrestricted paths

---

26. Reliability and Disaster Recovery

Define service objectives.

Example targets:

```
Storefront availability: 99.95%
Dashboard availability: 99.9%
Checkout success: 99.9% excluding provider outages
Webhook processing: under 60 seconds normally
RPO: 15 minutes
RTO: 2 hours
```

Implement:

- Multi-zone application deployment
- Database high availability
- Point-in-time recovery
- Encrypted backups
- Backup retention policy
- Daily backup verification
- Monthly restore drills
- Disaster recovery environment

- Queue durability
- Provider outage fallback
- Graceful degradation
- Status page
- Incident runbooks
- On-call ownership

Restore testing is mandatory. A backup that has never been restored is not verified.

---

27. Observability

Implement OpenTelemetry-compatible tracing.

Every request and job should carry:

```
request_id
trace_id
organization_id
actor_id
route
operation
```

Monitor:

- Error rates
- Latency
- Database performance
- Queue depth
- Job failures
- Webhook failures
- Payment mismatches
- Checkout failures
- Realtime connection failures

- Domain provisioning failures
- Storage scan failures
- AI failures
- Export failures
- Tenant isolation violations

Alert on:

- Error-rate spikes
- Payment webhook backlog
- Failed renewals
- Database saturation
- Queue dead letters
- Cross-tenant authorization errors
- Suspicious login activity
- Certificate failures
- Backup failures
- Elevated refund activity
- Unusual export volume

---

28. CI/CD Pipeline

Every pull request must run:

1. Formatting
2. Linting
3. Type checking
4. Unit tests
5. Integration tests
6. Database migration validation

7. RLS tests
8. Storage isolation tests
9. Realtime authorization tests
10. Dependency audit
11. Secret scan
12. Static analysis
13. Container scan
14. Build verification
15. Preview deployment
16. Dynamic application scan
17. End-to-end tests

Production deployment flow:

```
Pull request
  -> CI checks
  -> Review
  -> Preview deployment
  -> Security checks
  -> Staging deployment
  -> Migration validation
  -> Smoke tests
  -> Approval
  -> Production canary
  -> Health verification
  -> Progressive rollout
  -> Post-deployment monitoring
```

Database migrations must be backward-compatible during rolling deploys.

Use expand-and-contract migrations:

```
Add new column
  -> Deploy code supporting old and new schema
  -> Backfill data
```

```
-> Switch reads and writes
-> Remove old column later
```

Never couple a deployment to an irreversible migration without a rollback strategy.

---

29. Testing Program

29.1 Unit Tests

Cover:

- Money calculations
- Tax calculations
- Discount rules
- Order transitions
- Booking availability
- Slug generation
- Configuration schemas
- Role permissions
- AI action authorization
- Notification preferences
- Export formatting
- Retry behavior

29.2 Integration Tests

Cover:

- Authentication
- Organization creation
- Memberships
- RLS
- Product creation

- Checkout
- Payment webhooks
- Subscription transitions
- File uploads
- CSV imports
- Exports
- Domain provisioning
- Event publishing
- Job processing

29.3 End-to-End Tests

Test:

```
Create account
  -> Create business
  -> Select business mode
  -> Select template
  -> Import product
  -> Review import
  -> Publish storefront
  -> Customer places order
  -> Payment webhook confirms payment
  -> Owner receives notification
  -> Owner fulfills order
  -> Customer receives receipt
  -> Subscription renews or is canceled
```

29.4 Tenant Isolation Matrix

For Tenant A and Tenant B, assert:

- A cannot read B products
- A cannot update B orders
- A cannot delete B customers
- A cannot access B storage

- A cannot subscribe to B realtime channels
- A cannot retrieve B records through guessed IDs
- A cannot export B data
- A cannot invoke AI tools against B
- A cannot process B background jobs
- A cannot access B through admin APIs
- Suspended tenants cannot create orders
- Deleted users cannot retain access
- Expired sessions cannot access data

---

30. Compliance and Governance

Implement operational support for:

- Privacy policy acceptance
- Terms acceptance
- Cookie preferences
- Consent records
- Data subject requests
- Data export
- Data deletion
- Retention policies
- Vendor and subprocessor registry
- Data-processing agreements
- Security incident response
- Access reviews
- Audit logs
- Regional tax configuration

Legal review is required before launch. Requirements depend on:

- Customer location
- Tenant location
- Business type
- Payment flow
- Data-processing roles
- Applicable privacy laws
- Communications channels
- Tax obligations

Do not represent the platform as legally compliant solely because technical controls exist.

---

31. Data Export and Deletion

Exports must include:

- Products
- Categories
- Orders
- Customers
- Addresses
- Inventory
- Bookings
- Promotions
- Configuration
- Audit records where appropriate
- Media files

Export requirements:

- Reauthentication
- Permission checks
- Job-based generation
- Signed download links
- Expiration
- Audit log
- Completion notification
- Rate limiting
- Tenant isolation

Deletion workflow:

```
Request deletion
  -> Reauthenticate
  -> Confirm scope
  -> Start retention review
  -> Apply legal holds
  -> Disable tenant
  -> Delete or anonymize records
  -> Delete storage
  -> Revoke sessions
  -> Revoke domains
  -> Remove queued jobs
  -> Record completion
```

---

32. Platform Administration

Admin tools must include:

- Tenant search
- Tenant status
- Subscription status
- Domain status
- Usage

- Failed jobs
- Failed webhooks
- Billing reconciliation
- Security events
- Support tickets
- AI action review
- Export and deletion requests
- Feature flags
- Rate-limit controls

Support impersonation must:

- Require explicit support permission
- Require a reason
- Display a visible impersonation banner
- Limit available actions
- Log every action
- Expire automatically
- Prevent sensitive operations unless separately authorized
- Be reviewable by platform administrators

---

33. Product Delivery Phases

Phase 0: Discovery and Governance

Deliver:

- Capability map
- Business rules
- Tenant lifecycle
- Pricing and billing rules

- Threat model
- Data classification
- Compliance assessment
- Architecture decision records
- Design system foundations
- API conventions
- Incident response plan
- Support model
- Data retention policy
- Backup and recovery objectives

Exit criteria:

- Core workflows defined
- Security boundaries approved
- Billing behavior documented
- Legal review initiated
- Production ownership assigned

Phase 1: Engineering Foundation

Build:

- Monorepo
- TypeScript standards
- CI/CD
- Environment management
- Authentication
- Organizations
- Memberships
- Roles

- Tenant resolver
- Database migrations
- RLS framework
- Audit logs
- Feature flags
- Observability
- Error handling
- Job infrastructure
- Outbox infrastructure

Exit criteria:

- Two tenants coexist safely
- Cross-tenant tests pass
- Preview deployments work
- Failed jobs are observable
- Migrations are repeatable
- Logs include request and tenant context

Phase 2: Tenant Runtime and Publishing

Build:

- Configuration system
- Template registry
- Theme tokens
- Draft and preview releases
- Publish and rollback
- Hostname resolution
- Subdomain provisioning
- Custom domain verification

- SSL automation
- SEO framework
- CDN strategy
- Storefront caching

Exit criteria:

- Tenant can publish without developer action
- Rollback works
- Template switching preserves data
- Domain verification cannot be bypassed
- Cache invalidation works correctly

Phase 3: Catalog and Customer Foundation

Build:

- Catalog
- Categories
- Products
- Variants
- Images
- Services
- Customers
- Customer accounts
- Search
- Import review system
- CSV export

Exit criteria:

- Catalog operations are fully usable

- Customer records are tenant-isolated
- Imports are reviewable and reversible
- Search does not leak records

Phase 4: Checkout and Orders

Build:

- Cart
- Checkout
- Tax
- Discounts
- Pickup and delivery configuration
- Orders
- Fulfillments
- Receipts
- Order events
- Customer notifications

Exit criteria:

- Full checkout succeeds
- Price tampering is rejected
- Duplicate checkout is safe
- Order transitions are auditable
- Fulfillment works from the dashboard

Phase 5: Payments and Billing

Build:

- Payment provider integration
- Payment intents

- Webhooks
- Refunds
- Trial subscriptions
- Billing portal
- Subscription state machine
- Renewal reminders
- Past-due restrictions
- Reconciliation

Exit criteria:

- Payment webhooks are idempotent
- Duplicate events are safe
- Failed renewals produce correct restrictions
- Refunds are authorized and auditable
- Cancellation behavior is tested

Phase 6: Dashboard Operations

Build:

- Dashboard home
- Role-aware navigation
- Orders
- Catalog
- Customers
- Inventory
- Bookings
- Growth
- Appearance
- Settings

- Team
- Billing
- Exports
- Notifications

Exit criteria:

- Owners can operate daily
- Staff see only permitted workflows
- Empty states are actionable
- Dashboard supports mobile and desktop
- Export data is complete

Phase 7: Business Modules

Implement in controlled order:

1. Inventory
2. Bookings
3. Quotes
4. Promotions
5. Reviews
6. Delivery zones
7. Staff scheduling
8. Customer messaging
9. Marketing automation
10. Advanced analytics

Each module requires:

- Data model
- Permissions

- UI
- API
- Notifications
- Export support
- Audit events
- Billing impact
- RLS tests
- Documentation
- Recovery behavior

Phase 8: AI and Ingestion

Build:

- AI assistant
- Approved tool registry
- AI action contracts
- Review queue
- Catalog extraction
- Duplicate detection
- Description generation
- SEO generation
- Policy drafting
- Customer reply drafting
- Usage limits
- Cost controls
- Prompt and tool audit logs
- Human approval workflows

Exit criteria:

- AI cannot execute unauthorized actions
- AI output remains unpublished until review where required
- Every mutation is attributable
- Actions are reversible
- Model failures do not corrupt data

Phase 9: Reliability and Security Hardening

Build and verify:

- Full RLS suite
- Storage isolation
- Realtime authorization
- Dynamic security scans
- External penetration test
- Rate-limit tests
- Load tests
- Backup restore drills
- Disaster recovery environment
- Observability dashboards
- Incident runbooks
- On-call process
- Status page

Exit criteria:

- No unresolved critical or high-risk findings
- Recovery objectives are tested
- Cross-tenant isolation is independently verified
- Payment reconciliation is operational
- Restore drills are documented

Phase 10: Controlled Beta

Use a controlled cohort large enough to test operational diversity.

Track:

- Onboarding completion
- Time to publish
- Import accuracy
- Time to first order
- Checkout failure rate
- Payment failure rate
- Job failure rate
- Support burden
- Feature adoption
- Trial conversion intent
- Cancellation reasons
- Domain failures
- Realtime reliability

Do not expand features based only on individual requests. Prioritize repeated operational problems.

Phase 11: Public Launch

Launch only when:

- Legal documents are complete
- Support workflows operate
- Billing support is ready
- Status page is active
- Backups are verified
- Restore tests are complete

- Security review is complete
- Monitoring alerts are configured
- Data export works
- Cancellation works
- Incident ownership is assigned
- Provider failure behavior is tested
- Deployment rollback is verified
- Customer communications are approved

---

34. Production Readiness Checklist

Architecture

- [ ] Tenant resolver implemented
- [ ] Tenant context cannot be browser-controlled
- [ ] RLS enabled on all exposed tenant tables
- [ ] Storage policies tested
- [ ] Realtime policies tested
- [ ] Background jobs tenant-aware
- [ ] Outbox and inbox patterns implemented
- [ ] Domain events versioned

Authentication

- [ ] Session revocation works
- [ ] MFA enforced for sensitive roles
- [ ] Reauthentication implemented
- [ ] Recovery flow tested
- [ ] Suspicious-login monitoring enabled

Payments

- [ ] Webhook signatures verified
- [ ] Event deduplication implemented
- [ ] Payment state derived from webhooks
- [ ] Refund idempotency implemented
- [ ] Reconciliation jobs operational
- [ ] Billing transitions audited

Data

- [ ] Data exports implemented
- [ ] Deletion workflow implemented
- [ ] Retention policy documented
- [ ] Backup encryption enabled
- [ ] Restore drill completed
- [ ] Storage isolation verified

Reliability

- [ ] Health checks
- [ ] Readiness checks
- [ ] Liveness checks
- [ ] Queue monitoring
- [ ] Dead-letter handling
- [ ] Provider outage handling
- [ ] Rollback procedure
- [ ] Disaster recovery procedure

Security

- [ ] Secret scan

- [ ] Dependency scan
- [ ] Container scan
- [ ] SAST
- [ ] DAST
- [ ] Penetration test
- [ ] Threat model reviewed
- [ ] Security incident process documented

Operations

- [ ] On-call rotation
- [ ] Incident runbooks
- [ ] Status page
- [ ] Support escalation process
- [ ] Billing reconciliation process
- [ ] Audit review process
- [ ] Access review schedule

---

35. Final Release Gate

The final release should be blocked unless the following end-to-end scenario passes repeatedly in staging and production-like environments:

```
User creates account
  -> Creates organization
  -> Selects business mode
  -> Configures brand
  -> Imports catalog
  -> Reviews imported records
  -> Publishes storefront
  -> Customer visits custom or platform domain
  -> Customer adds item or books service
  -> Customer completes checkout
  -> Payment provider confirms through webhook
```

```
  -> Order or appointment becomes operational
  -> Tenant receives realtime dashboard update
  -> Staff fulfills or manages the transaction
  -> Customer receives confirmation
  -> Billing trial transitions correctly
  -> Renewal succeeds or fails safely
  -> Tenant can cancel
  -> Tenant can export data
  -> Tenant can request deletion
```

At every step, verify:

- Correct tenant isolation
- Correct permissions
- Correct audit events
- Correct notification behavior
- Correct retries
- Correct rollback behavior
- Correct observability
- Correct failure handling

---

36. Final Architecture Principle

The completed platform should consist of:

```
Stable shared runtime
+
Versioned tenant configuration
+
Approved modular capabilities
+
Strict multi-layer authorization
+
RLS-backed tenant isolation
+
Auditable AI actions
```

```
+
Reliable billing state machine
+
Transactional commerce workflows
+
Tenant-scoped realtime events
+
Portable customer data
+
Observable background processing
+
Automated deployment and rollback
+
Tested disaster recovery
```

The ultimate production standard is not the number of templates, modules, or AI features. The system is ready when:

- A business can publish independently.
- Customers can reliably purchase, book, or submit inquiries.
- Owners can operate the business from the dashboard.
- Staff permissions are correctly enforced.
- Payments and billing reconcile predictably.
- Failed jobs and provider outages recover safely.
- Data can be exported and deleted.
- Every important action is traceable.
- No tenant can access another tenant’s data.
- Production operations can detect, contain, and recover from incidents.